



南京大學

本科畢業論文

学 院	智能软件与工程学院
专 业	软件工程（智能化软件）
题 目	实践中的理论最优哈希表
年 级	2022 学 号 221900323
学生姓名	纪泽操
指导教师	刘明谋 职 称 副教授
提交日期	2026 年 5 月 20 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：实践中的理论最优哈希表

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

哈希表是计算机领域最基础的核心数据结构，近六十年来，如何在操作时间与存储空间之间取得最优权衡，一直是数据结构方向的核心研究难题。本文以《On the Optimal Time/Space Tradeoff for Hash Tables》为研究基础，对论文中的核心算法原理、工程实现细节进行梳理，并尝试简化实现论文中描述的哈希系统。

论文中包含三部分核心工作，第一是复现 K - kick Tree 球槽映射及分层驱逐这种核心算法；第二是设计并实现 MiniArray 与本地查询路由两类紧凑元数据结构，完成变长路由信息的低开销存储与常数时间访问；第三则是在此基础上搭建增强开放寻址哈希表整体架构，实现插入、删除、查找、动态扩容及商压缩优化机制，完整复现论文设计逻辑。

最终实现常数时间查询，插入与删除时间高概率可控制，而且具备动态扩容与缩容的精简哈希表系统。经实验数据的结果来看，该系统再现了原论文当中有关时间和空间兼顾的特点，证实了这种较为理想的哈希结构在实际工程中有得以落实的可能性，进而给高性能精简哈希表的设计及其应用赋予一定的参照。

关键词：哈希表；时间空间权衡；探针复杂度；增强开放寻址；k-kick tree；元数据编码；动态可扩容；succinct 数据结构

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: On the Optimal Time/Space Tradeoff for Hash Tables

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

Hash tables are the most fundamental core data structures in the field of computer science. For nearly sixty years, how to achieve the optimal balance between operation time and storage space has been a central research problem in the study of data structures. This paper is based on "On the Optimal Time/Space Tradeoff for Hash Tables," and it organizes the core algorithm principles and engineering implementation details presented in the paper, while attempting to simplify the implementation of the hash system described.

The paper contains three core contributions. The first is reproducing the key algorithm of K-kick Tree slot mapping and hierarchical eviction; the second is the design and implementation of two types of compact metadata structures, MiniArray and local query routing, which enable low-overhead storage of variable-length routing information and constant-time access; the third is building an enhanced open-addressing hash table architecture based on this, implementing insertion, deletion, lookup, dynamic resizing, and quotient compression optimization mechanisms, fully reproducing the design logic of the paper.

The final implementation achieves constant-time queries, and the insertion and deletion times are highly probable to be controlled, while providing a streamlined hash table system with dynamic expansion and contraction. According to experimental results, this system replicates the time and space tradeoff characteristics described in the original paper, demonstrating that this relatively ideal hash structure can be realized in practical engineering, thus providing a reference for the design and application of high-performance streamlined hash tables.

KEYWORDS: hash tables; time–space tradeoff; probe complexity; enhanced open addressing; k-kick tree; metadata encoding; dynamic expansion; succinct data structures

目 录

目 录	IV
第一章 引言	1
1.1 研究背景	1
1.2 研究目标	1
第二章 问题模型与理论基础	3
2.1 开放寻址中的两个代价	3
2.2 k-kick 树的核心机制	4
2.3 从理论量到系统指标	5
2.4 本章小结	5
第三章 系统设计	6
3.1 总体目标	6
3.1.1 Facility: 设施级路由与分布管理	6
3.1.2 Cubby: 微观存储单元	7
3.1.3 Slot 与 MetaEntry: 从落槽到键恢复	7
3.2 紧凑构件设计	8
3.2.1 MiniArray	8
3.2.2 Local Query Router	8
3.3 商压缩流程	8
3.4 动态重建与扩缩容	9
3.5 主操作路径	9
3.6 本章小结	10
第四章 系统实现	11

4.1	工程结构	11
4.2	参数与可逆置换	11
4.3	核心数据结构	12
4.3.1	HashTable 与 TableState	12
4.3.2	Facility 与 Cubby	12
4.3.3	MetaEntry 与 商压缩	12
4.4	MiniArray 与 Router 的工程实现	13
4.5	插入、查询和删除实现	13
4.6	后台重建与扩缩容	14
4.7	工程差异说明	14
第五章	系统验证与实验设计	15
5.1	验证目标	15
5.2	功能正确性测试	15
5.3	性能基准	16
5.4	指标与数据库观测	16
5.5	结构不变量验证	17
5.6	可视化与结果呈现	17
5.7	本章小结	18
第六章	总结与展望	19
6.1	工作总结	19
6.2	不足之处	19
6.3	后续工作	20
6.4	结束语	20
	参考文献	21
	致 谢	22
	附录 A 数据模式与运行配置补充	23
A.1	关系模式（由 ensure_analytics_ddl 创建）	23

A.1.1	snapshot_meta	23
A.1.2	facility、cubby、slot_snapshot、tier_stat.	23
A.1.3	otsh_op_metrics（操作级事实表）	25
A.1.4	otsh_test_session_log	25
A.2	清空语义	26
A.3	运行配置（环境变量）	26

第一章 引言

1.1 研究背景

哈希表一般被当作“平均 $O(1)$ ”的数据结构看待，不过工程系统关注的并非仅仅是平均时间，键值存储，缓存索引以及数据库执行引擎里的哈希表常常要应对三种约束：其一，查询路径需短；其二，内存占用应接近信息论下界；其三，插入与删除不可因局部冲突造成无法控制的长尾延误。开放寻址哈希表把数组槽位用以保存元素，其指针开销小，缓存局部性也好，但在高负载因子时，冲突会延长探针序列，而且删除和动态扩容会破坏简单线性探测的局部结构。

Bender 等人在《On the Optimal Time/Space Tradeoff for Hash Tables》中给出一类最优时间/空间折中哈希表构造^[1]。该构造把开放寻址中的冲突处理当作“球与槽”的在线分配问题来抽象，用探针复杂度来表现每个键需另外记录多少位信息，用交换成本来表现一次更新时移动已有元素的代价，其核心结论可大致理解为：若准许插入和删除执行 $O(k)$ 次受控移动，则可将每个元素的额外空间从较高的对数量级缩减到 $O(\log^{(k)} n)$ 量级，此结论属于理论算法范畴，但它给工程执行赋予了明晰的方向，即空间的节省并非源于简单地压缩某个字段，而是得益于寻址，路由，元数据和后台重建之间的协同设计。

本文工作按照此种思路来开展，旨在遵照论文里的理论，利用 C++ 去完成一套既可运行又可观测的原型，再经由检测和指标确认关键行为是否达到理论目的，所关注的问题在于：理论当中的 k - kick 分层驱逐，Local Query Router, MiniArray, Facility，商业压缩以及动态重建，怎样结合成一个可以执行插入，查询，删除和导出结构并且记录指标的系统。

1.2 研究目标

本文的目标可以分为三个层次。

第一，整理论文中的基本原理，包括“球与槽”模型、 k - kick 树、MiniArray、

Local Query Router、商压缩以及维护动态不变量，保持每一个 facility 中仅有一个 tail cubby 不满。

第二，系统用 C++20 来做到核心哈希表，并把 HashTable 当作对外的接口，内部按照 Facility -> Cubby -> Slot 这种分层方式来组织架构，各个 Facility 需要守护一个路由器数组 D ，一些多层级的 Cubby 以及一条尾巴；而每个 Cubby 则要保留负载槽，空闲槽位树还有可变长度的元数据；商压缩相关的资料会经由 MetaEntry 进行编码；至于扩缩容，则是依靠双表迁移并结合后台预算慢慢达成目标。

第三，创建验证路径，其上可采集如下指标：利用 router_probe_steps 来观测路由路径；以 kick_count 及移动次数来观测交换成本；凭借 meta_bits 对元数据开销予以估算；经由 tier/cubby 快照来观测分布不变量；通过 otsh_tests 和 otsh_perf 分别对功能正确性以及热路径性能实施验证。

第二章 问题模型与理论基础

2.1 开放寻址中的两个代价

开放寻址哈希表把元素直接放在数组槽中。对一个键 x ，哈希函数给出探针序列

$$h_1(x), h_2(x), h_3(x), \dots$$

查询时沿序列依次寻找，插入时选择某个空槽。若负载较低，这一路径很短；若负载接近满表，冲突会让路径变长。《On the Optimal Time/Space Tradeoff for Hash Tables》把这个过程抽象成“球与槽”的在线分配：槽位数为 n ，任意时刻最多有 n 个球，每个球只能被放在自己探针序列中的某一项，更新操作允许移动已经放入的球。

在该模型中，我们注意到探针复杂度和交换成本。

- **探针复杂度：**若球 x 最终放在探针序列的第 i 个位置，其复杂度记为 $\Theta(1 + \log i)$ 。直观上，查询若要直接跳到这个位置，系统需要知道“它用了第几个候选槽”，记录该索引需要约 $\log i$ 位。因此，平均探针复杂度可以看作额外元数据空间的代理量。
- **交换成本：**一次插入或删除可能把已有元素踢到新位置。被移动元素的数量反映了更新操作的代价，也会影响实际系统中延迟。

传统开放寻址常把冲突处理放在探针序列本身，例如线性探测、二次探测或双重哈希^[2]。论文的思路更主动：允许更新时重新安排少量已有元素，用有限交换换取更短的平均探针复杂度。本文后续实现中的 `kick_count`、移动次数和路由步数，正是对这两个理论量的具体观测。

2.2 k-kick 树的核心机制

k-kick 树不是必须按指针实际存储的一棵树，而是一套对数组槽位进行分层划分和优先级驱逐的规则，逻辑上的管理方式。设槽位总数为 n ，逻辑上把数组划为 $k + 1$ 层：第 0 层是整个数组，第 1 层划为大小约 $(\log n)^6$ 的 bin，第 2 层可缩小到 $(\log \log n)^6$ 量级，依此类推。第 i 层 bin 的大小记为

$$s_i = \Theta((\log^{(i)} n)^6).$$

越深的 bin 越小，也越接近一个元素的“理想局部位置”，优先级越高。

每个球 x 在插入前由随机性确定三类信息。第一，它选择一个叶层 cubby，并由此确定所有祖先 cubby $g_0(x), g_1(x), \dots, g_k(x)$ 。第二，它得到一个最高允许深度 $s(x)$ ，该值决定优先级，深层元素优先留在更小的局部区域。第三，它的探针序列按从深到浅的顺序遍历这些 cubby：依次 $g_k(x)$ 、 $g_{k-1}(x)$ 、...、 $g_0(x)$ 。

插入过程可以描述为“寻找一个未饱和 cubby，然后把更低优先级元素向上层推”。对新球 x ，算法从较深层开始检查其祖先 cubby。若某个 cubby 有空槽，则直接放入。若 cubby 已满但未饱和，说明其中至少有一个不应该占据该层优先位置的低优先级球；算法将其踢出，让 x 占据该槽。被踢出的球再到自己允许的较浅层继续寻找位置，必要时继续踢出其他低优先级球。这一过程最多形成 $k + 1$ 级级联。

这一过程的关键在于层数受 k 控制。每一次踢出都会把问题推向更浅层，同一层不会无限反复，因此一次插入最多形成 $k + 1$ 级级联。删除操作也可以通过从 tail 或其他局部结构迁移元素补洞，保持类似的受控移动。理论分析给出的结果是：在适当随机假设和不变量维护下，更新成本为 $O(k)$ ，平均探针复杂度为 $O(\log^{(k)} n)$ 量级^[1]。

对本文实现来说，这个结论有两层意义。第一，系统需要记录每次插入和删除到底移动了多少元素，用来判断工程路径是否出现异常长链。第二，系统要把“深层局部优先”的思想落实到 Cubby、tier 和 tail 的组织中，而不是仅依赖普通开放寻址的首次空槽策略。

2.3 从理论量到系统指标

理论分析中的探针复杂度和交换成本不能直接等同于某个 C++ 计时结果。实际延迟还受到缓存、锁、HTTP 编解码、MySQL 写入和操作系统调度影响。因此本文在实验章采用“可观测代理量”的方式验证原理实现。

- 路由步数对应查询从键到 (cubby, slot) 的查找成本;
- `kick_count` 与移动次数对应插入、删除中的交换成本;
- `meta_bits` 与 Router 编码长度对应额外元数据空间;
- tier 计数、cubby 容量和 tail 状态对应分布不变量;
- 延迟分位数用于观察工程实现是否出现不可接受的长尾。

2.4 本章小结

本章从开放寻址的动态冲突问题出发，说明了探针复杂度与交换成本的含义，并介绍了 k-kick 树通过分层 cubby 和级联驱逐获得 $O(k)$ 更新代价的基本机制。下一章将在此基础上讨论完整系统设计：如何用 MiniArray、Local Query Router、Facility、商压缩和后台重建把这些理论部件连接为可运行的哈希表。

第三章 系统设计

3.1 总体目标

系统需要支持初始化、插入、查询、删除、批量加载、结构遍历和指标采集；同时保留理论结构中的关键接口，使实验能够观察到探针、踢出、元数据位数、tier 分布和扩缩容事件。

从复杂度目标看，理想模型希望查询在局部路由与元数据访问均为常数时间时达到 $O(1)$ ，插入和删除以 $O(k)$ 次交换为主要代价，总额外空间接近 $O(n \log^{(k)} n)$ 比特^[1]。当前系统更强调可解释和可验证：它保留 Facility、Cubby、Router、MiniArray、商压缩、tail 与后台重建等结构，但在若干微观部件上采用工程化实现。因此本文后续讨论会区分“理论目标”和“当前实现”。

系统采用 Facility $\rightarrow$ Cubby $\rightarrow$ Slot 的三层结构。这个结构方便把宏观容量管理、局部插入策略和每槽元数据分开处理。

3.1.1 Facility：设施级路由与分布管理

Facility 是全表的宏观分片。设全局容量参数为 N ，单个设施管理约 $K = \text{polylog } N$ 规模的局部元素。对键 x 先计算可逆置换 $y = \pi(x)$ ，再从 $g^*(x) = y \bmod N$ 中拆出设施编号 r 和设施内桶编号 b 。系统访问第 r 个 Facility 的 $D[b]$ ，由该局部路由器返回目标 Cubby 和槽位。

一个 Facility 内部包含三类状态：

- D : 长度为 K 的路由器数组，每个元素管理一个小规模键集合到 (Cubby, slot) 的映射；
- 多 tier cubby 集合：不同 tier 表示不同容量尺度，用于近似理论中的分层 cubby；
- tail：唯一允许非满的 cubb。

tail 的存在使“除 tail 外尽量保持满”的不变量更容易维护。插入优先进入 tail; 删除非 tail 元素后, 可从 tail 搬迁一个元素补洞。这样做可以减少普通 cubby 长期半空造成的空间浪费, 也为后续重建提供较稳定的输入, 为装填因子可达到 1 提供了可能。

3.1.2 Cubby: 微观存储单元

Cubby 是实际保存元素的基本单元。逻辑上, 一个 Cubby 对应理论中的局部 k-kick 实例; 工程上, 它包含以下状态:

- slots: 槽数组, 保存商压缩后的 payload, 而不是简单保存完整键;
- occupied: 记录已占用槽位, 便于迁移、删除和结构导出;
- FreeSlotTree: 用于快速寻找空槽;
- MiniArray meta: 按槽保存变长 MetaEntry 编码。

在理论构造中, Cubby 内部应按 k-kick 树的分层优先级执行插入。当前系统保留踢出链和移动计数, 用有限随机探针和局部搬迁近似该行为。这样可以让插入、删除、查询路径完整运行, 并能通过 kick_count 与移动次数观察更新成本, 但它不是论文中 packed 微观结构的逐行复现。

3.1.3 Slot 与 MetaEntry: 从落槽到键恢复

每个槽位不仅保存 payload, 还需要保存恢复原键所需的信息。系统把这些信息压入 MetaEntry, 主要包括:

- delta: 偏好槽 $g_I(x)$ 与实际槽 i 的差值;
- mid_bits: 从 Cubby 内位置拼回设施内路由低位所需的中间位;
- router_bits: 路由辅助信息;
- insert_bits: 插入过程相关标签。

MetaEntry 的意义在于把“元素为什么能在这个槽被找到”压缩成少量可编码字段。查询时, 系统并非只根据键重新线性探测, 而是先经 Router 定位到槽, 再利用 payload 与 MetaEntry 恢复通过反函数 $\pi^{-1}(y)$ 得到的原键, 最后用逆置换校验原键。

3.2 紧凑构件设计

3.2.1 MiniArray

MiniArray 用来保存变长元数据。理想结构可理解为单字宽节点组成的 packed B-树：叶子保存实际比特，内部节点保存偏移与枢轴，配合查找表实现均摊常数时间访问和更新。它解决的问题是：如果每个槽的元数据长度不同，普通定长数组会浪费空间，普通动态字节串又难以快速定位。

当前系统采用较直接的工程版本：用连续 bit buffer 保存所有条目，用 offsets 和 bitlens 定位每个槽的编码。更新某个槽时允许重新打包整个 Cubby 的元数据。由于 Cubby 规模被限制在 polylog 范围内，这个代价在原型系统中可接受；同时 bits_total 能直接作为空间统计的输入，方便实验章估计元数据开销。

3.2.2 Local Query Router

Local Query Router 负责少量键的快速定位。设计上，它把键哈希成随机二进制串，构造压缩前缀 Trie，只保存能区分键集合的最短前缀，并在叶子处记录目标 Cubby 与 slot。Trie 的编码长度随局部元素数增长，理论上可以结合查表做到常数时间跳转。

当前系统的 Router 使用 64 位哈希串上的二叉 trie，并处理哈希碰撞桶。插入和删除之后允许重建 trie 与编码长度统计。这样的策略牺牲了理论中最细粒度的 $O(1)$ 更新保证，但代码简单、状态清楚，且局部元素数较小时开销可控。实验上，router_probe_steps 和 bits_total 可用来观察路由成本和编码规模。

3.3 商压缩流程

商压缩的目标是避免在槽中重复存储完整键。设 π 是 64 位可逆置换， $g^*(x) = \pi(x) \bmod N$ 。 $g^*(x)$ 的低位已经用于选择 Facility、路由桶和 Cubby 内偏好位置，因此这些位可以部分由寻址上下文隐含携带。Cubby 的 slots 保存的是去掉低 $\log N$ 位后的 quotient payload；MetaEntry 保存把实际槽位还原为偏好槽、再

拼回 $g^*(x)$ 所需的信息。

查询流程因此分为两段。第一段是定位：由 $\pi(x)$ 的低位进入对应 Facility 和 Router，得到目标 Cubby 与 slot。第二段是校验：从 slot 读取 payload 和 MetaEntry，根据设施编号、Cubby 容量、slot 索引和 delta/mid_bits 重建 $\pi(x)$ ，再调用 π^{-1} 得到原键。若恢复出的键与查询键一致，则返回命中。这个设计把一部分键位从显式存储转移到寻址过程和元数据中，是系统接近信息论空间目标的主要来源。

3.4 动态重建与扩缩容

论文的构造要求不同 tier 的 cubby 数量维持在目标分布附近。设计文档中使用近似目标

$$t_j \approx \frac{(\log^{(j)} n)^2}{(\log^{(j+1)} n)^2}.$$

当某一级 cubby 过少时，可以把更高一级 cubby 拆分；当某一级 cubby 过多时，可以把若干低级 cubby 合并。重建本身需要搬迁元素，如果一次性完成会造成延迟尖峰，因此系统把重建任务放入调度队列，每次更新只消耗固定预算。

全局扩缩容采用双表迁移。活动表负责新请求；当元素数超过负载上界或低于容量下界时，系统创建新 TableState，旧表保留为迁移来源，随后按设施逐步搬迁元素。这样做避免一次扩容搬动所有元素，也便于通过 `resize_start` 和 `resize_finish` 等指标观察迁移过程。

3.5 主操作路径

插入路径。系统先计算 $y = \pi(x)$ ，拆出设施和桶编号 r 和 b ，在 $D[b]$ 中检查键是否已存在。若不存在，将元素放入该 Facility 的 tail Cubby，选择槽位并写入 quotient payload 与 MetaEntry，然后更新空闲槽结构和 Router。如果 tail 满，则创建新的 tail 或触发局部重建；全局规模变化后再检查是否需要扩容。

查询路径。查询同样从 $\pi(x)$ 出发定位 Facility 和 Router。Router 返回 Cubby 与 slot 后，系统读取 slot payload 和 MetaEntry，反函数 $x' = \pi^{-1}(y)$ 。该路径的关键不在于遍历长探针序列，而在于局部路由和元数据能否把目标槽直接指出来。

删除路径。删除先复用查询定位目标槽，再清空 slot、MetaEntry 和 Router 记录。如果删除发生在非 tail cubby，系统从 tail 迁移一个元素填补空洞，保持普通 cubby 的满载倾向。删除后若规模过低，则触发缩容迁移。

3.6 本章小结

本章给出了当前系统的总体设计。Facility 负责宏观路由和分布管理，Cubby 负责局部存储和受控移动，Slot 与 MetaEntry 负责商压缩下的键恢复，MiniArray 和 Router 负责紧凑元数据与局部定位。下一章将结合代码文件说明这些设计如何具体实现，以及哪些部分属于工程化近似。

第四章 系统实现

4.1 工程结构

当前仓库使用 CMake 管理 C++20 工程，主要生成三个可执行文件。第一个是 `otsh_backend`，负责启动 HTTP 服务、连接存储层并对外提供实验接口；第二个是 `otsh_tests`，用于执行内存哈希表回归测试，并在条件允许时写入测试会话日志；第三个是 `otsh_perf`，不链接 MySQL，专门测量纯内存插入、查询和删除路径。HTTP 层使用 `cpp-httpplib`^[3] 和 `nlohmann/json`^[4]，持久化层使用 MySQL/MariaDB 客户端接口^[5]。

核心哈希表的对外接口定义在 `include/ht.h`。接口包括 `init`、`insert`、`query`、`erase`、`bulk_load`、`state`、`visit_structure` 和 `pi_of`。这些接口覆盖了论文中的基本操作，也为实验章中的结构快照和指标采集提供入口。真正的数据结构实现集中在 `src/ht.cpp` 的 `HashTable::Impl` 中，其他头文件主要承载可组合的小模块。

4.2 参数与可逆置换

初始化参数由 `TableParams` 描述，包含目标规模 n 、折中参数 k 、负载因子、随机种子和存储配置。系统将容量 N 取为不小于需求的 2 的幂，并派生局部规模 K 与 Facility 数量。这样做一方面简化了位运算拆分，另一方面让 $g^*(x) = \pi(x) \bmod N$ 可以直接通过低位截取得到。

`PermutationHash` 实现 64 位可逆置换。它把输入拆成高低两个 32 位半区，使用三轮 Feistel 结构混合，轮密钥由种子经 `splitmix64` 派生。Feistel 结构的好处是无论轮函数是否可逆，整体置换都可以反向计算。系统在查询时利用 π^{-1} 做键恢复校验，在结构导出时也可用 `pi_of` 记录键的置换像。

4.3 核心数据结构

4.3.1 HashTable 与 TableState

HashTable::Impl 内部维护当前活动表状态。活动表包含元素数 n 、容量 N 、局部规模 K 和 `std::vector<Facility>`。当需要扩容或缩容时，实现会创建新表，并把旧表保存在可选状态中，通过迁移进度逐步搬迁元素。迁移完成后释放旧表。这一策略对应设计章的双表迁移，避免一次更新承担全部 rehash 成本。

操作结果结构也带有验证字段。InsertResult 和 DeleteResult 中包含 kick_count 和 router_probe_steps, QueryResult 中包含 router_probe_steps。这些字段不是业务返回值，而是为了把理论中的交换成本、路由路径转化为可记录指标。

4.3.2 Facility 与 Cubby

include/otsh/facility.h 中的 Facility 保存多 tier cubby、tail 和路由器数组 D。其中 tiers 是按层组织的二维容器，元素为 `std::unique_ptr<Cubby>`；tail 指向当前唯一允许非满的 cubby；D 是 `std::vector<Router>`，负责设施内局部定位。

include/otsh/cubby.h 中的 Cubby 保存 tier、capacity、size、槽数组、占用索引、空闲槽树和 MiniArray 元数据。槽数组类型为 `std::optional<uint64_t>`，存放的是商压缩后的 payload。占用索引用于快速遍历已有元素，空闲槽树用于插入时寻找可用位置，元数据数组用于保存每个槽的恢复信息。

4.3.3 MetaEntry 与商压缩

include/otsh/meta_entry.h 和 src/meta_entry.cpp 实现 MetaEntry 的构造、编码、解码与键恢复。核心函数包括 quotient_payload、make_meta_entry、encode_meta_entry、decode_meta_entry 和 reconstruct_gx_pi。

插入时，系统根据 $\pi(x)$ 、设施编号、Cubby 容量和实际 slot 生成 MetaEntry。其中 delta 保存偏好槽与实际槽的差，mid_bits 保存拼接低位所需的信息。

查询时，系统先通过 Router 得到 slot，再解码该槽元数据，用 payload 和上下文重建 $\pi(x)$ 。如果逆置换得到的键与查询键一致，则查询成功。这个过程把商压缩从公式变成了可执行路径。

4.4 MiniArray 与 Router 的工程实现

MiniArray 位于 `include/otsh/mini_array.h`。它用 `buf_` 保存连续 bit buffer，用 `offsets_` 和 `bitlens_` 定位每个条目。读取时按 bit 拷贝出对应条目；更新时重新计算偏移并重打包。该实现没有采用论文级 packed B-树和查表加速，因此单次更新不是严格 $O(1)$ 。不过，Cubby 本身是小规模结构，重打包代价在原型中可接受，并且 `bits_total` 可以直接用于空间观测。

Router 位于 `include/otsh/router.h` 和 `src/router.cpp`。它把键经 `splitmix64` 映射为 64 位随机串，在二叉 trie 中保存从键到 (`CubbyPtr`, `slot`) 的映射。实现中保留碰撞桶，避免 64 位哈希串完全相同导致信息丢失。插入和删除后调用重建逻辑，重新计算 trie 与编码 bit 数。该实现的重点是正确表达 Local Query Router 的定位语义，并提供 `bits_total` 和路由步数，而不是完成理论中的半字查表优化。

4.5 插入、查询和删除实现

插入时，`HashTable::Impl` 先计算置换值并定位 Facility 与路由桶。如果 Router 中已有该键，则返回 `inserted=false`，保持重复插入幂等。若为新键，则在 tail Cubby 中寻找槽位，必要时执行有限踢出和搬迁；随后写入 payload、编码 MetaEntry、更新 FreeSlotTree 与 Router，并记录移动次数、路由步数和元数据位数。插入完成后，系统检查 tail 状态、重建任务和扩容条件。

查询实现复用同样的设施和桶定位逻辑。Router 返回目标后，系统读取槽位 payload 和元数据，调用恢复函数重建置换像并验证键。若恢复失败、slot 为空或键不一致，则返回未命中。由于 Router 直接给出位置，查询路径不会退化为扫描整个 Cubby。

删除实现先定位键，随后清理 Router 记录、slot 和 MetaEntry。如果删除位置不在 tail 中，系统尝试从 tail 取出元素迁入空槽，以维持非 tail cubby 的满载不

变量。删除后同样更新指标，并检查是否需要缩容。该过程对应理论中删除后局部重排的工程化版本。

4.6 后台重建与扩缩容

`include/otsh/rebuild.h` 中的 `RebuildScheduler` 是一个任务队列，每次操作消耗有限预算执行部分任务。`src/ht.cpp` 中的重建逻辑根据 `tier` 分布决定拆分或合并 `cubby`，并通过调度器分摊执行。进程内指标会记录 `rebuild_down` 和 `rebuild_up` 事件，便于观察分布维护是否频繁触发。

扩缩容逻辑内联在 `HashTable::Impl` 中。负载超过上界时启动扩容，元素数低于阈值时启动缩容。迁移过程中，新表与旧表同时存在，查询和删除需要考虑两边状态；迁移完成后旧表释放。指标中的 `resize_start` 和 `resize_finish` 可用于确认迁移是否按预期闭环。

4.7 工程差异说明

当前实现与论文理想结构存在三个主要差异。第一，`k-kick` 树没有以独立完整类实现全部分层饱和逻辑，而是在 `HashTable::Impl` 的插入路径中以有限踢出和 `tail` 机制近似。第二，`MiniArray` 没有实现 `packed B-树` 和查表更新，而采用重打包 `bit buffer`。第三，`Router` 在插入和删除后允许重建 `trie`，而不是严格维护常数时间局部更新。这些差异不会影响本文对系统功能和实现路径的描述，但会影响对性能结果的解释。因此，第五章只把测试指标作为“原理实现的验证证据”，不把原型运行时间直接声称为论文理论上界的实证证明。

第五章 系统验证与实验设计

5.1 验证目标

本文验证的重点不是重新证明 OTSH 理论，而是检查当前系统是否把设计文档中的关键原理落实到了可运行路径中。验证目标分为四类：第一，基本语义正确，即插入后的键可查，删除后的键不可查，重复插入不产生重复元素；第二，操作代价可观测，即路由步数、踢出次数、移动次数和延迟能够被记录；第三，结构状态可导出，即 Facility、Cubby、Slot 和 tier 分布可以形成快照；第四，实验可复现，即相同规模、种子和参数下能够得到可比较的 JSON 或数据库记录。

由于当前原型在 Router、MiniArray 和 k-kick 细节上存在工程化简化，实验结论应表述为“实现行为验证”和“指标趋势观察”，而不是对原论文渐近上界的严格实证证明。

5.2 功能正确性测试

otsh_tests 是系统的自动化回归入口。该程序直接调用 HashTable，不经过 HTTP 层，因此更适合检查核心数据结构本身。测试流程包括：

- 初始化哈希表，并检查返回状态与表状态中的 N 、Facility 数量是否有效；
- 使用固定随机种子生成唯一键集合，执行批量插入，要求每个新键返回 `inserted=true`；
- 对全部已插入键执行查询，要求返回 `found=true`；
- 随机删除一半键，要求删除操作成功；
- 再次查询已删除键，要求返回未命中；
- 对剩余键重复插入，要求返回 `inserted=false`，验证幂等语义。

测试程序结束时输出一行 JSON，其中包含 `all_passed`、通过和失败数量、每个 case 的名称与结果，以及 `ops_insert`、`ops_query`、`ops_delete`、

insert_moved_total、delete_moved_total 等指标摘要。若 MySQL 可用，结果追加到 otsh_test_session_log；否则写入本地测试日志。这样，功能验证既能由进程退出码判断，也能留下可审计记录。

5.3 性能基准

otsh_perf 用于隔离算法热路径。它不链接 MySQL，也不经过 HTTP JSON 编解码，主要测量内存中的插入、顺序命中查询、随机命中查询、删除和混合查询。程序支持命令行参数 n_insert、n_rand_queries、table_n_hint，并通过环境变量 OTSH_SEED 控制随机种子。

每个阶段记录操作数、墙钟时间、平均延迟、p50、p95 和 p99。人类可读结果输出到 stderr，单行 JSON 输出到 stdout，便于脚本收集到 perf/results/。这些数据可以用于回答三类问题：规模增加时查询延迟是否平稳；插入和删除是否出现异常尾部；改变参数 k 或初始容量时，移动次数与延迟是否发生可解释变化。

5.4 指标与数据库观测

操作级指标以 SqlMetricRow 形式产生，核心字段包括 operation_type、probe_count、kick_count、latency_ns、cubby_tier 和 snapshot_id。存储层先把指标放入内存缓冲，再批量写入 otsh_op_metrics，避免高频操作逐条落库造成过大扰动。

这些字段对应第二章中的理论量。probe_count 可用于观察路由成本，kick_count 可用于观察更新中的交换成本，latency_ns 用于描述工程延迟，cubby_tier 可用于区分不同层级的行为差异。典型查询包括按操作类型统计平均和分位延迟、按 tier 统计平均踢出次数、按快照比较不同参数下的路由步数。

结构快照由 HashTable::visit_structure 驱动。系统把 Facility、Cubby、Slot 和 tier 聚合写入对应表。facility 记录 tail 所在 cubby，cubby 记录 tier、capacity、size 和 is_tail，slot_snapshot 记录槽位是否占用及 $\pi(x)$ 形式的 key hash，tier_stat 记录每个 Facility 在各 tier 上的 cubby 数量。

这些表可用于检查 tail 是否唯一、非 tail cubby 是否接近满载、tier 数量是否偏离目标分布。

5.5 结构不变量验证

围绕设计章中的不变量，本文建议采用以下检查方式。

- **tail 唯一性**。在每个 Facility 的快照中统计 `is_tail=true` 的 cubby 数量，应为 1 或处于可解释的初始化状态；
- **容量利用率**。对非 tail cubby 计算 `size/capacity`，检查是否保持较高利用率；
- **tier 分布**。将 `tier_stat` 中的实际 cubby 数与 t_j 的目标趋势比较，观察重建是否能把分布拉回合理范围；
- **移动成本**。对 `kick_count` 和 `insert_moved_total` 做直方图，检查是否存在频繁长链；
- **恢复正确性**。通过查询命中和删除后未命中间接验证商压缩恢复路径；必要时结合结构快照中的 $\pi(x)$ 值进行抽查。

5.6 可视化与结果呈现

实验结果可以从两类来源生成图表。一类是 `otsh_perf` 输出的 JSON，适合画不同规模下的延迟分位数和吞吐；另一类是 HTTP 分析接口或 MySQL 表，适合画平均探针、踢出次数、tier 分布、cubby 利用率和尾延迟曲线。可视化工具可以选用 ECharts^[6] 或其他图表库，本文不限定具体前端实现。

论文呈现实验结果时，应把图表解释与系统实现联系起来。例如，若删除阶段的移动次数高于查询阶段延迟变化，应回到 tail 补洞和迁移逻辑解释；若某组参数下重建事件频繁，应结合 tier 分布和容量利用率分析。这样能避免只罗列性能数字，也能突出本文关注的“原理实现和验证”。

5.7 本章小结

本章给出了系统验证方案。功能测试证明基本语义，性能基准隔离内存热路径，HTTP 接口验证服务化运行，数据库指标和结构快照用于观察理论量在工程中的代理表现。所有实验都围绕实现是否忠实表达设计文档中的核心机制展开，而不是用单次运行数字替代严格理论证明。

第六章 总结与展望

6.1 工作总结

本文围绕最优时间/空间折中哈希表的工程实现展开。理论部分从开放寻址的冲突问题出发,使用“球与槽”模型解释探针复杂度和交换成本,并介绍 k-kick 树如何通过分层 cubby、随机优先级和级联驱逐把更新代价控制在 $O(k)$ 量级。系统设计部分进一步说明,单靠一个探针函数无法达到目标,必须把局部驱逐、紧凑元数据、局部路由、商压缩和后台重建组合起来。

系统以 HashTable 为核心接口,内部按 Facility、Cubby 和 Slot 分层组织;使用可逆置换 π 支持商压缩和键恢复;使用 Router 管理键到槽位的局部映射;使用 MiniArray 保存变长元数据;使用 FreeSlotTree 管理空槽;使用 RebuildScheduler 和双表迁移处理 tier 重建与扩缩容。服务层通过 HTTP 暴露初始化、插入、查询、删除、统计、结构导出和参数实验接口,存储层通过 MySQL 记录键、指标和结构快照。

验证部分没有把原型性能简单等同于论文理论上界,而是建立了更适合当前系统的证据链。otsh_tests 检查核心语义,otsh_perf 测量无数据库热路径,HTTP 接口验证服务化流程,Metrics 与 otsh_op_metrics 记录路由步数、踢出次数、移动次数、延迟和重建事件,结构快照则用于观察 Facility、Cubby、Slot 和 tier 分布。

6.2 不足之处

当前系统与论文中的完整理论结构存在差距。

- k-kick 树没有完全实现论文级分层饱和判定和严格优先级驱逐,而是在插入路径中以有限踢出、tail 和局部搬迁近似;
- MiniArray 采用连续 bit buffer 与重打包实现,尚未实现 packed B-树、查找

表加速和严格均摊常数更新；

- Router 使用二叉 trie 与重建策略表达局部路由语义，尚未实现理论中的紧凑编码常数时间更新；
- 实验章强调验证方法和可采集指标，若要形成最终定量结论，还需要在固定硬件、固定编译参数和多组随机种子下补充完整运行数据。

6.3 后续工作

后续优化可以沿三条线推进。第一条是算法结构补全：把 k-kick 树的层级饱和、优先级和级联驱逐从当前插入路径中抽出，形成独立模块，并与 tier 重建共享不变量。第二条是紧凑结构优化：用 packed B-树或分块 bitset 改写 MiniArray，用增量维护的压缩 Trie 改写 Router，减少更新后的整体重建。第三条是实验体系完善：补齐不同 n 、 k 、负载因子和随机种子下的批量实验，把延迟分位数、元数据位数、移动次数和 tier 分布整理成稳定图表。

此外，当前结构快照已经能导出 Facility、Cubby 和 Slot 信息，后续可以接入离线分析脚本或可视化页面，把一次实验中的操作指标和结构状态放在同一时间线上观察。这会有助于区分算法本身的代价、HTTP/数据库 I/O 的代价以及扩缩容迁移造成的短时波动。

6.4 结束语

本文完成的工作可以概括为：把一套以空间最优为目标的哈希表理论结构，拆解为可实现的系统模块，并为每个关键原理保留验证入口。虽然当前原型还没有达到论文中所有微观结构的最优复杂度，但它已经能够展示论文思想在工程系统中的主要形态，也为后续继续完善算法细节和实验数据打下基础。

参考文献

- [1] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[EB/OL]. (2021-11-04) [2026-05-02]. <https://arxiv.org/abs/2111.00602>. arXiv: 2111.00602.
- [2] KNUTH D E. The Art of Computer Programming, Volume 3: Sorting and Searching[M]. 2nd ed. Reading, MA: Addison-Wesley, 1998.
- [3] HIRAOKA Y. cpp-httpplib[EB/OL]. 2026 [2026-04-27]. <https://github.com/yhirose/cpp-httpplib>.
- [4] LOHMANN N. JSON for Modern C++[EB/OL]. 2026 [2026-04-27]. <https://github.com/nlohmann/json>.
- [5] Oracle. MySQL Reference Manual[A/OL]. 2026 [2026-04-27]. <https://dev.mysql.com/doc/>.
- [6] Apache Software Foundation. Apache ECharts Documentation[EB/OL]. 2026 [2026-04-27]. <https://echarts.apache.org/>.

致 谢

行文至此，落笔为终。感谢遇到的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。

附录 A 数据模式与运行配置补充

A.1 关系模式（由 ensure_analytics_ddl 创建）

下列表在首次 storage->open 成功时按需创建。主键表名由 TableParams::mysql_table 指定（默认 otsh_keys），单列 k BIGINT UNSIGNED PRIMARY KEY，用于持久化当前键集（与内存结构同步策略以 /api/init 的 reset 与插入/删除路径为准）。

A.1.1 snapshot_meta

为避免 Windows 下 lower_case_table_names 导致列名冲突，规模字段在库中命名为 init_param_n、table_n、table_k：

```
CREATE TABLE IF NOT EXISTS `snapshot_meta`(  
  `id` BIGINT NOT NULL AUTO_INCREMENT,  
  `snapshot_tag` VARCHAR(64),  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  `init_param_n` BIGINT NOT NULL,  
  `table_n` BIGINT UNSIGNED NOT NULL,  
  `table_k` BIGINT UNSIGNED NOT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB;
```

A.1.2 facility、cubby、slot_snapshot、tier_stat

```
CREATE TABLE IF NOT EXISTS `facility`(  
  `id` INT NOT NULL,  
  `snapshot_id` BIGINT NOT NULL,
```

```

    `tail_cubby_id` INT NOT NULL,
    PRIMARY KEY (`id`,`snapshot_id`),
    KEY `ix_fac_snap` (`snapshot_id`)
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS `cubby`(
    `id` INT NOT NULL,
    `snapshot_id` BIGINT NOT NULL,
    `facility_id` INT NOT NULL,
    `tier` INT NOT NULL,
    `capacity` INT NOT NULL,
    `size` INT NOT NULL,
    `is_tail` TINYINT(1) NOT NULL,
    PRIMARY KEY (`id`,`snapshot_id`),
    KEY `ix_cub_snap` (`snapshot_id`)
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS `slot_snapshot`(
    `id` BIGINT NOT NULL AUTO_INCREMENT,
    `snapshot_id` BIGINT NOT NULL,
    `cubby_id` INT NOT NULL,
    `slot_index` INT NOT NULL,
    `occupied` TINYINT(1) NOT NULL,
    `key_hash` BIGINT UNSIGNED NULL,
    `probe_length` INT NOT NULL,
    PRIMARY KEY (`id`),
    KEY `ix_slot_snap` (`snapshot_id`),
    KEY `ix_slot_cubby` (`snapshot_id`,`cubby_id`)
) ENGINE=InnoDB;

```

```
CREATE TABLE IF NOT EXISTS `tier_stat` (
  `snapshot_id` BIGINT NOT NULL,
  `facility_id` INT NOT NULL,
  `tier` INT NOT NULL,
  `cubby_count` INT NOT NULL,
  PRIMARY KEY (`snapshot_id`, `facility_id`, `tier`)
) ENGINE=InnoDB;
```

A.1.3 otsh_op_metrics (操作级事实表)

```
CREATE TABLE IF NOT EXISTS `otsh_op_metrics` (
  `id` BIGINT NOT NULL AUTO_INCREMENT,
  `snapshot_id` BIGINT NOT NULL,
  `operation_type` ENUM('insert', 'query', 'delete') NOT NULL,
  `probe_count` INT NOT NULL,
  `kick_count` INT NOT NULL,
  `latency_ns` BIGINT NOT NULL,
  `cubby_tier` INT NOT NULL,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `ix_met_snap` (`snapshot_id`)
) ENGINE=InnoDB;
```

实现中缓冲满 1000 条或强制 flush 时批量插入，单条 INSERT 语句最多拼接 500 行值。

A.1.4 otsh_test_session_log

供 otsh_tests 在跑完回归后追加一行 JSON:

```
CREATE TABLE IF NOT EXISTS `otsh_test_session_log` (
  `id` BIGINT NOT NULL AUTO_INCREMENT,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
`body` MEDIUMTEXT NOT NULL,  
PRIMARY KEY (`id`)  
) ENGINE=InnoDB;
```

A.2 清空语义

`/api/init` 在 `reset=true` 时调用 `DELETE` 清空上述分析表及主键表（对不存在的历史表名忽略错误），以便重复实验。

A.3 运行配置（环境变量）

- 服务: `BIND_HOST`、`PORT`、`LOG_FILE`;
- 自动化测试 `otsh_tests`: `OTSH_MYSQL_HOST`、`OTSH_MYSQL_PORT`、`OTSH_MYSQL_USER`、`OTSH_MYSQL_PASSWORD`、`OTSH_MYSQL_DATABASE`、`OTSH_MYSQL_TABLE`、`OTSH_TEST_LOG`;
- 基准 `otsh_perf`: `OTSH_SEED` 等（见 `perf/bench.cpp` 注释）。